

PHASE 3- PROJECT DESIGN

Project Title: POCKETSMART AI

Your Smart Budget & Recommendation Assistant

Name: Government Arts and Science College, Manur.

College Code: MSU142

Team ID: 6ab21524d329d50f0db6120b

Team Leader: Esakki Aruna S **NM ID:**55055706AAD495BADB04C8C4D3B478B8

Team Member: Utchi Makali **NM ID:** 0BBEDB52C642EC0B0BA4CB4B22DE9E2D

Team Member: Kombu Madathi **NM ID:** 86EE18ABFB36AEF83784FA01234C5948

Team Member: S Ulagammal **NM ID:** B59DED29E07C4F78BCF442535560F984

1. Introduction

The Project Design Phase converts the requirements of PocketSmart AI into a structured technical design. It describes the system architecture, major modules, database structure, user interface, processing flow, and AI recommendation workflow. The design is intended to keep the application simple, secure, modular, and easy to maintain.

2. Design Objectives

- Create a simple and user-friendly budgeting interface.
- Separate the presentation, application logic, database, and recommendation components.
- Provide accurate calculations for income, expenses, budgets, and savings.
- Design a recommendation workflow that uses available user financial data.
- Protect personal financial information through authentication and controlled access.
- Allow future features to be added without redesigning the complete system.

3. Proposed System Architecture

PocketSmart AI follows a layered architecture consisting of the Presentation Layer, Application/Backend Layer, AI Recommendation Layer, and Database Layer.

- **Presentation Layer** – Provides the web or mobile interface through which users enter data and view results.
- **Application Layer** – Handles authentication, transaction processing, budget calculations, savings tracking, and report generation.
- **AI Recommendation Layer** – Analyzes relevant user data and produces personalized budgeting or purchase suggestions.
- **Database Layer** – Stores user accounts, transactions, budgets, savings goals, and recommendation-related information.

4. Major System Modules

4.1 User Management Module – Handles registration, login, authentication, profile information, and access control.

4.2 Income & Expense Module – Allows users to add, edit, delete, categorize, and view financial transactions.

4.3 Budget Management Module – Creates monthly and category-wise budgets and calculates remaining budget amounts.

4.4 Savings Goal Module – Stores savings targets and displays progress toward each goal.

4.5 Spending Analysis Module – Processes transaction history to identify spending categories, trends, and budget deviations.

4.6 AI Recommendation Module – Uses budget status, spending patterns, goals, and planned purchase information to generate relevant suggestions.

4.7 Dashboard & Reporting Module – Displays summaries, charts, alerts, and monthly financial reports.

5. Data Flow Design

1. User logs into the application.

2. User enters income, expenses, budgets, savings goals, or planned purchases.
3. The backend validates and processes the input.
4. Validated data is stored in the database.
5. The analysis component calculates spending and budget status.
6. The AI recommendation component receives relevant summarized data.
7. Recommendations, alerts, and reports are returned to the dashboard.
8. The user reviews the results and updates financial information when necessary.

6. Database Design

The proposed database can contain the following main entities:

Users: user_id, name, email, password_hash, created_at

Income: income_id, user_id, source, amount, date, description

Expenses: expense_id, user_id, category_id, amount, date, description

Categories: category_id, user_id, category_name

Budgets: budget_id, user_id, category_id, amount_limit, period

SavingsGoals: goal_id, user_id, goal_name, target_amount, current_amount, target_date

Recommendations: recommendation_id, user_id, recommendation_type, message, created_at

7. Database Relationships

- One user can have multiple income records.
- One user can have multiple expense records.
- One category can be associated with multiple expense records.
- One user can create multiple budgets and savings goals.
- One user can receive multiple recommendations.
- User ID acts as the primary relationship between personal financial records.

8. User Interface Design

- Login/Register Screen – Provides account creation and authentication.
- Dashboard – Shows balance, total income, total expenses, budget status, savings progress, and key alerts.
- Add Transaction Screen – Allows users to record income or expenses.
- Budget Screen – Allows creation and monitoring of monthly and category budgets.
- Savings Screen – Displays goals, target amounts, current progress, and remaining amounts.
- AI Assistant Screen – Provides a conversational or form-based interface for budget and purchase questions.
- Reports Screen – Displays monthly summaries and charts.

9. Dashboard Design

The dashboard should provide a quick overview of the user's financial position. A possible layout includes:

- Total Income card.
- Total Expenses card.
- Remaining Budget/Balance card.
- Savings Progress card.
- Category-wise expense chart.
- Monthly spending trend chart.
- Recent transactions list.
- AI recommendation and alert section.

10. AI Recommendation Design

The recommendation component is designed to convert financial information into understandable suggestions. The system should use relevant data such as income, regular expenses, budget limits, spending history, savings targets, and planned purchase amount.

- Budget Recommendation – Suggest a spending limit for selected categories.
- Saving Recommendation – Suggest an amount or adjustment that supports the user's savings target.
- Spending Insight – Identify categories where spending has increased.

- Purchase Support – Compare a planned purchase with the user's available budget.
- Budget Alert – Highlight situations where spending is close to or above a defined limit.

11. Recommendation Processing Flow

1. Collect relevant user financial data.
2. Validate and summarize the data.
3. Calculate current budget and spending status.
4. Identify relevant patterns or conditions.
5. Apply recommendation rules and/or AI processing.
6. Generate a clear recommendation.
7. Display the recommendation with supporting budget information.

12. Security Design

- Use password hashing rather than storing plain-text passwords.
- Restrict users to their own financial records.
- Validate user input before storing or processing it.
- Use secure session or token-based authentication.
- Avoid exposing sensitive financial information in unnecessary logs or responses.
- Apply appropriate access controls to administrative functions.

13. Error Handling Design

- Display validation messages for missing or invalid inputs.
- Prevent negative or invalid values where they are not logically allowed.
- Handle database failures without exposing technical details to users.
- Provide clear messages when AI recommendations cannot be generated.
- Allow users to correct incorrectly entered transactions.

14. Proposed Technology Stack

- Frontend: HTML, CSS, JavaScript or a suitable web/mobile framework.
- Backend: Python with Flask or Django.

- Database: MySQL or SQLite.
- AI/Recommendation: Python-based rules/recommendation engine and/or an AI service.
- Charts: Chart.js or another suitable visualization library.
- Development Tools: Visual Studio Code or another suitable IDE.

15. Design Advantages

- Modular architecture makes the project easier to develop and maintain.
- Separate modules allow individual features to be tested independently.
- A centralized dashboard provides quick access to important financial information.
- The AI layer adds personalized recommendations beyond basic expense tracking.
- The design can be extended later with features such as receipt scanning, voice input, or multi-language support.

16. Project Design Summary

The PocketSmart AI design combines a user-friendly interface, structured financial data management, budget and savings calculations, spending analysis, and an AI-assisted recommendation component. The proposed architecture separates major responsibilities into independent layers and modules, providing a clear foundation for implementation and testing.